Select the most appropriate option from among the given four options. [2 marks each]

1. What will be the output for the code below? ..... (B)

```
a = 0
b = 0
for i in range(10):
    if i%2 == 0:
        a += 1
    else:
        b += a
print(b)
```

(A) 20 (B) 15

(C) 10 (D) 5

2. What will be the output for the code below? ..... (A B)

```
p = 10
s = 0
while p > 0:
    if p > 5:
        s += 2
    else:
        break
    p -= s
print(p)
```

(A) 2 (B) 4

(C) 6 (D) 8

3. What will be the output for the code below? ..... (B)

```
points = [1, -1, 2, -2, 3, -3, 4, -4, 5, -5]
mapper = dict([(i, points[i]) for i in
range(len(points))])
for x in '622':
    score = mapper[int(x)]
print(score)
```

(A) -2 (B) 2

(C) 4 (D) 8

4. What is the result of *a* for the code below? ..... (D)

```
a = 20
for i in range(10, 20):
    a += i
```

i	a (20)
10	30
11	41
12	53
13	66
14	80
15	95
16	111
17	128
18	146
19	165

(A) 55 (B) 145

(C) 155 (D) 165

5. What is the result of *b* for the code below? ..... (B X C)

```
a = 'abcbcabacabccabb'
b = a.find('a', 8)
```

(A) -1 (B) 0

(C) 9 (D) 13

starting index

6. Which string(s) will be in the output list after running the code below? .... (C)

```
import re
p = re.compile(r'a[bcd*]e', re.M)
match = p.findall('abe abbe acdde ace adbe acde')
print(match)
```

should be outside [ ]

(i) abbe
(ii) ace
(iii) acdde
(iv) adbe ?.

(A) (i), (ii) and (iv) (B) (i) and (iv)

(C) (i), (ii), (iii) and (iv) (D) (ii)

7. What code produces the output below? ..... (B)

```
[0, 1, 2, 6, 8, 10]
```

(A) [i if i>3 else i*2 for i in range(6)]

(B) [i if i<3 else i*2 for i in range(6)]

(C) [i*2 if i>3 else i for i in range(6)]

(D) [i*2 if i<3 else i for i in range(6)]

8. What code produces the output below? ..... (B X A)

```
'e!x!a!m'
```

(A) '!'.join('exam')

(B) '!'.join('exa') + 'm'

(C) 'exam'.join('!')

(D) 'exa'.join('!') + 'm'

(9) [4 marks] Create a Python class named Course that has the attributes `name`, `term` and `special`. The method `set_as_special` modifies one of the attributes by setting it to True. The other method, `describe_course`, returns a string which takes into account the aforementioned attributes.

Fill in the blanks below so that the entire code can run without any errors.

```
##################################################
######                Start of code             ######
##################################################

class ____Course()_____:
    def __init__(self, name, term, special):
        self.name = name
        self.term = term
        self.special = False

    def set_as_special(self):
        ___self.special = True___

    def describe_course(self):
        if __self.special == True:__
            return "The special course {} will be in {}.".format(self.name, self.term)
        else:
            return "The regular course {} will be in {}.".format(self.name, self.term)

course = Course('Advanced Coke Appreciation', 'Nov 2023')
course.set_as_special()
assert course.describe_course() == 'The special course Advanced Coke Appreciation will be
in Nov 2023.'

##################################################
######                End of code               ######
##################################################
```

not advisable since many things can trigger a True

1. if self.special
2. if self.special is True → also not advisable because this checks if the obj is exactly the same, so might accidentally trigger False